

```
=====
BLOG STORAGE - EXPRESS + AZURE BLOB STORAGE
All project files combined
=====
```

```
=====
FILE: README.md
=====
```

```
# Blog Storage (Express + Azure Blob Storage)
```

Each blog post is stored as a single JSON blob at `posts/{id}.json` inside a container (default name `blog-posts`). No database needed.

```
## Setup
```

```
```bash
npm install
cp .env.example .env
fill in AZURE_STORAGE_CONNECTION_STRING (or ACCOUNT + KEY) in .env
npm start
```
```

The container is created automatically on first request if it doesn't exist.

```
### Local testing without a real Azure account
```

Run the [Azurite emulator](https://learn.microsoft.com/azure/storage/common/storage-use-azurite) and point `AZURE\_STORAGE\_CONNECTION\_STRING` at it:

```
```bash
npx azurite --silent --location ./azurite-data
```
```

```
...
```

```
AZURE_STORAGE_CONNECTION_STRING=DefaultEndpointsProtocol=http;AccountName=devstoreaccount1;AccountKey=Eby8vdm02xN0cqFlqUwJPLlmEtlCDXJ1OUzFT50uSRZ6IFsuFq2UVERCz4I6tq/K1SZFPT0tr/KBHBeksoGMGw==;BlobEndpoint=http://127.0.0.1:10000/devstoreaccount1;
```
```

```
API
```

Method	Path	Body	Description
GET	/posts	-	List all posts
GET	/posts/:id	-	Get one post
POST	/posts	{ title, body, author? }	Create a post
PUT	/posts/:id	{ title?, body?, author? }	Update a post
DELETE	/posts/:id	-	Delete a post

```
Notes / production considerations
```

- **Listing scales linearly** – `listPosts` downloads every blob's content to build the list. Fine for hundreds of posts; for a larger blog, store lightweight metadata (title, date, slug) in a separate `index.json` blob or move to Table Storage / Cosmos DB for the index, keeping Blob Storage for the body content.
- **Concurrency**: `createPost` uses `ifNoneMatch: '\*'` so it fails if a blob with that id already exists. `updatePost` currently does a plain overwrite; add an `ifMatch` ETag condition if you need optimistic concurrency control on edits.
- **Auth**: this sample has no request authentication – add your own middleware (JWT, session, etc.) before exposing write routes publicly.
- For production Azure deployments, consider `DefaultAzureCredential` (from `@azure/identity`) instead of account keys, so you can use

Managed Identity rather than storing a key in `.env`.

```
=====
FILE: package.json
=====
```

```
{
 "name": "blog-storage",
 "version": "1.0.0",
 "description": "Express blog API backed by Azure Blob Storage",
 "main": "src/server.js",
 "type": "commonjs",
 "scripts": {
 "start": "node src/server.js",
 "dev": "node --watch src/server.js"
 },
 "dependencies": {
 "@azure/storage-blob": "^12.24.0",
 "dotenv": "^16.4.5",
 "express": "^4.19.2",
 "uuid": "^9.0.1"
 }
}
```

```
=====
FILE: .env.example
=====
```

```
Option A: connection string (also works with Azurite local emulator)
AZURE_STORAGE_CONNECTION_STRING=

Option B: account name + key (used only if connection string is not set)
AZURE_STORAGE_ACCOUNT=
AZURE_STORAGE_KEY=

Container where post JSON blobs are stored
AZURE_STORAGE_CONTAINER=blog-posts

PORT=3000
```

```
=====
FILE: src/server.js
=====
```

```
require('dotenv').config();
const express = require('express');
const postsRouter = require('./routes/posts');

const app = express();
app.use(express.json());

app.use('/posts', postsRouter);

app.get('/health', (req, res) => res.json({ status: 'ok' }));

// Central error handler
app.use((err, req, res, next) => {
 console.error(err);
 res.status(err.statusCode || 500).json({ error: err.message || 'Internal server error' });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
 console.log(`Blog API listening on port ${PORT}`);
});
```

```

});

=====
FILE: src/blobStorage.js
=====
const { BlobServiceClient, StorageSharedKeyCredential } = require('@azure/storage-blob');

const CONTAINER_NAME = process.env.AZURE_STORAGE_CONTAINER || 'blog-posts';
const PREFIX = 'posts/';

let containerClientPromise = null;

function buildBlobServiceClient() {
 // Preferred: full connection string (works for local Azurite emulator too)
 if (process.env.AZURE_STORAGE_CONNECTION_STRING) {
 return BlobServiceClient.fromConnectionString(process.env.AZURE_STORAGE_CONNECTION_STRING);
 }

 // Alternative: account name + key
 const accountName = process.env.AZURE_STORAGE_ACCOUNT;
 const accountKey = process.env.AZURE_STORAGE_KEY;
 if (accountName && accountKey) {
 const credential = new StorageSharedKeyCredential(accountName, accountKey);
 return new BlobServiceClient(
 `https://${accountName}.blob.core.windows.net`,
 credential
);
 }

 throw new Error(
 'Missing Azure Storage credentials. Set AZURE_STORAGE_CONNECTION_STRING, or AZURE_STORAGE_ACCOUNT + AZURE_STORAGE_KEY.'
);
}

async function getContainerClient() {
 if (!containerClientPromise) {
 containerClientPromise = (async () => {
 const blobServiceClient = buildBlobServiceClient();
 const containerClient = blobServiceClient.getContainerClient(CONTAINER_NAME);
 await containerClient.createIfNotExists();
 return containerClient;
 })();
 }
 return containerClientPromise;
}

async function streamToString(readableStream) {
 const chunks = [];
 for await (const chunk of readableStream) {
 chunks.push(Buffer.isBuffer(chunk) ? chunk : Buffer.from(chunk));
 }
 return Buffer.concat(chunks).toString('utf-8');
}

function blobNameFor(id) {
 return `${PREFIX}${id}.json`;
}

async function listPosts() {
 const containerClient = await getContainerClient();
 const posts = [];
 for await (const blob of containerClient.listBlobsFlat({ prefix: PREFIX })) {

```

```

 const blockBlobClient = containerClient.getBlockBlobClient(blob.name);
 const download = await blockBlobClient.download();
 const content = await streamToString(download.readableStreamBody);
 posts.push(JSON.parse(content));
 }
 posts.sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt));
 return posts;
}

async function getPost(id) {
 const containerClient = await getContainerClient();
 const blockBlobClient = containerClient.getBlockBlobClient(blobNameFor(id));
 const exists = await blockBlobClient.exists();
 if (!exists) return null;
 const download = await blockBlobClient.download();
 const content = await streamToString(download.readableStreamBody);
 return JSON.parse(content);
}

async function createPost(post) {
 const containerClient = await getContainerClient();
 const blockBlobClient = containerClient.getBlockBlobClient(blobNameFor(post.id));
 const data = JSON.stringify(post, null, 2);
 await blockBlobClient.upload(data, Buffer.byteLength(data), {
 blobHTTPHeaders: { blobContentType: 'application/json' },
 // Fail if a post with this id already exists
 conditions: { ifNoneMatch: '*' },
 });
 return post;
}

async function updatePost(id, updatedPost) {
 const containerClient = await getContainerClient();
 const blockBlobClient = containerClient.getBlockBlobClient(blobNameFor(id));
 const exists = await blockBlobClient.exists();
 if (!exists) return null;
 const data = JSON.stringify(updatedPost, null, 2);
 await blockBlobClient.upload(data, Buffer.byteLength(data), {
 blobHTTPHeaders: { blobContentType: 'application/json' },
 overwrite: true,
 });
 return updatedPost;
}

async function deletePost(id) {
 const containerClient = await getContainerClient();
 const blockBlobClient = containerClient.getBlockBlobClient(blobNameFor(id));
 const result = await blockBlobClient.deleteIfExists();
 return result.succeeded;
}

module.exports = {
 listPosts,
 getPost,
 createPost,
 updatePost,
 deletePost,
};

```

```

=====
FILE: src/routes/posts.js
=====
const express = require('express');

```

```

const { v4: uuidv4 } = require('uuid');
const store = require('../blobStorage');

const router = express.Router();

// GET /posts - list all posts
router.get('/', async (req, res, next) => {
 try {
 const posts = await store.listPosts();
 res.json(posts);
 } catch (err) {
 next(err);
 }
});

// GET /posts/:id - get a single post
router.get('/:id', async (req, res, next) => {
 try {
 const post = await store.getPost(req.params.id);
 if (!post) return res.status(404).json({ error: 'Post not found' });
 res.json(post);
 } catch (err) {
 next(err);
 }
});

// POST /posts - create a post
router.post('/', async (req, res, next) => {
 try {
 const { title, body, author } = req.body;
 if (!title || !body) {
 return res.status(400).json({ error: 'title and body are required' });
 }

 const now = new Date().toISOString();
 const post = {
 id: uuidv4(),
 title,
 body,
 author: author || 'anonymous',
 createdAt: now,
 updatedAt: now,
 };

 await store.createPost(post);
 res.status(201).json(post);
 } catch (err) {
 // ifNoneMatch condition failure -> a post with this generated id somehow exists (extremely unlikely)
 if (err.statusCode === 409) {
 return res.status(409).json({ error: 'Post already exists' });
 }
 next(err);
 }
});

// PUT /posts/:id - update a post
router.put('/:id', async (req, res, next) => {
 try {
 const existing = await store.getPost(req.params.id);
 if (!existing) return res.status(404).json({ error: 'Post not found' });

 const { title, body, author } = req.body;
 const updated = {

```

```

 ...existing,
 title: title ?? existing.title,
 body: body ?? existing.body,
 author: author ?? existing.author,
 updatedAt: new Date().toISOString(),
 };

 await store.updatePost(req.params.id, updated);
 res.json(updated);
} catch (err) {
 next(err);
}
});

// DELETE /posts/:id - delete a post
router.delete('/:id', async (req, res, next) => {
 try {
 const deleted = await store.deletePost(req.params.id);
 if (!deleted) return res.status(404).json({ error: 'Post not found' });
 res.status(204).send();
 } catch (err) {
 next(err);
 }
});

module.exports = router;

```